

SPECTRA: Sparse Periodic Extraction for Efficient Retrieval-Augmented Forecasting

Anonymous Author(s)

Affiliation

email@example.com

Abstract

Time series forecasting is a cornerstone of modern decision-making systems. While deep learning models have achieved state-of-the-art performance, handling non-stationarity and distribution shifts remains a significant challenge. Retrieval-Augmented Forecasting (RAFT) has emerged as a promising method to address this by utilizing historical data as a non-parametric memory. However, to maximize recall, standard approaches typically employ “dense retrieval”, which involves searching through the entire history of overlapping windows. While effective, this strategy incurs substantial computational and memory costs, presenting a challenge for large-scale deployment. In this paper, we propose **SPECTRA** (Sparse Periodic Extraction for Conditional Time-series Retrieval Augmentation). Leveraging the inherent periodicity of time series, SPECTRA introduces a learnable convolutional operator to compress raw sequences into compact *periodic motifs*. By constructing a channel-independent, strided dictionary from these representations, we reduce the knowledge base size by two orders of magnitude ($100\times$). Experiments across seven well-established benchmarks demonstrate that SPECTRA matches or exceeds the predictive accuracy of dense retrieval while delivering up to a $10\times$ acceleration at inference time, closing the gap between retrieval accuracy and industrial scalability.

1 Introduction

Time series forecasting plays a pivotal role in critical infrastructure, including power grid management and financial market analysis. A fundamental challenge in these domains is *non-stationarity*, where the data-generating process evolves over time. Existing deep learning approaches, such as Transformers [Vaswani *et al.*, 2017] and MLP-based models [Zeng *et al.*, 2023a], encode temporal dynamics into a fixed set of model parameters. While effective under stationary assumptions, these parametric models often exhibit degraded performance when the test distribution deviates from the training distribution.

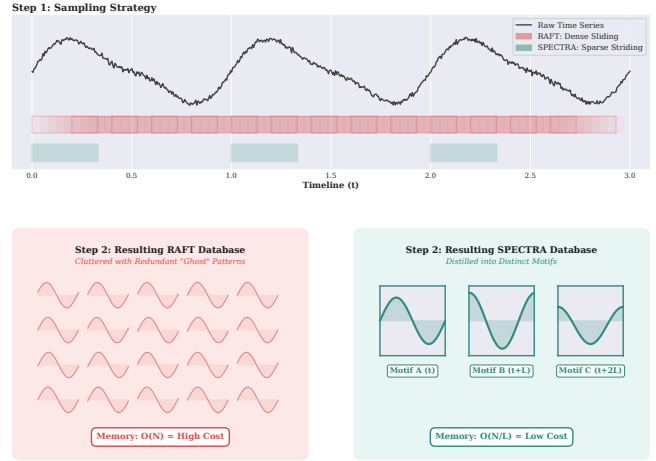


Figure 1: **Redundancy vs. Structure.** **Top:** RAFT (Red) densely samples overlapping windows, while SPECTRA (Teal) employs period-aligned striding. **Bottom:** RAFT’s database ($O(N)$) is cluttered with redundant “ghost” copies, whereas SPECTRA ($O(N/L)$) distills history into distinct motifs, maximizing efficiency without signal loss (N : dataset length, L : lookback window).

To mitigate this, the community has increasingly looked towards *Retrieval-Augmented Forecasting* (RAFT) [Han *et al.*, 2025]. Drawing inspiration from RAG in NLP, RAFT keeps the training, historical, data accessible during inference. For a given query window, the model retrieves similar historical sequences and uses their future values to guide the prediction. This “non-parametric” memory provides a robust mechanism to handle outliers by grounding predictions in actual historical precedents. The direct translation of text-based retrieval to time series is fraught with inefficiency. The standard RAFT method treats time series as a dense corpus of overlapping sliding windows. For a dataset of length N , it constructs a Key-Value store of size N . This “Dense Retrieval” strategy suffers from three fundamental limitations:

1. **Computational Bottleneck:** In high-frequency domains, searching a database of size $N = 10^6$ for every inference step is computationally prohibitive.
2. **Memory Overhead:** Storing dense floating-point representations for every timestep requires gigabytes of memory.

3. **Signal Redundancy:** Unlike natural language, time series are cyclically redundant. Therefore, storing every daily iteration of a load pattern captures mostly noise.

In order to address these challenges, we propose a retrieval framework that exploits the inherent *periodicity* of time series, focusing on *structural motifs* that capture recurring temporal patterns. Specifically, we introduce **SPECTRA** (Sparse Periodic Extraction for Conditional Time-series Retrieval Augmentation), which fundamentally rethinks knowledge base construction for time-series retrieval. Instead of storing raw and dense window sequences, SPECTRA employs a learnable **Spectral Extractor**, a convolutional operator, initialized to capture dominant frequencies. The resulting spectral representations are then used to construct a **Sparse Dictionary** by sampling with a large stride over non-overlapping windows. Crucially, this process goes beyond simple down-sampling. By leveraging the Nyquist–Shannon sampling theorem, we show that projecting time series into a periodic manifold enables lossless reconstruction of the predictive signal using only a minimal set of samples.

Overall, our contributions can be summarized as follows:

- **Concept:** We propose the first *sparse* retrieval framework for time series, shifting the paradigm from data-point retrieval to periodic-motif retrieval.
- **Architecture:** We design a lightweight, end-to-end trainable architecture that integrates spectral extraction, sparse storage, and adaptive fusion.
- **Results:** We achieve a $100\times$ reduction in storage and $10\times$ faster inference compared to RAFT. Experiments across seven benchmarks demonstrate that SPECTRA establishes a new state of the art, consistently outperforming both dense retrieval and advanced parametric baselines (e.g., PatchTST, DLinear), while matching the speed of non-retrieval models.

2 Related Work

2.1 Deep Time Series Forecasting

The landscape of time series forecasting has shifted significantly from statistical methods (ARIMA) to deep learning architectures. Early deep approaches relied on Recurrent Neural Networks (RNNs) and LSTMs. The introduction of the Transformer led to a surge in attention-based models designed to capture long-range dependencies. **Informer** [Zhou et al., 2021] and **Autoformer** [Wu et al., 2021] reduced the quadratic complexity of attention using ProbLogSparse and decomposition mechanisms, respectively. More recently, **PatchTST** [Nie and Liu, 2023] proposed segmenting time series into patches and utilizing channel-independent Transformers, achieving state-of-the-art results. Concurrently, simpler linear architectures like **DLinear** [Zeng et al., 2023b] and **TiDE** [Das et al., 2024] demonstrated that preventing overfitting is often more critical than architectural complexity. Despite their success, these “parametric” models share a common limitation: they compress historical knowledge into static weights. When the test distribution drifts (non-stationarity), these models struggle to adapt without retraining.

2.2 Frequency-Domain Modeling

Recognizing that time series are often governed by underlying cycles, several works incorporated frequency-domain analysis. **FEDformer** [Zhou et al., 2023] employed frequency-enhanced block attention, performing operations in the Fourier domain to capture global trends. **TimesNet** [Nie et al., 2023] transformed 1D time series into 2D tensors based on dominant periods, using 2D convolutions to capture intra-period and inter-period variations. **ETSformer** [Woo and Baek, 2023] utilized exponential smoothing and frequency attention. While these methods leveraged periodicity for feature extraction, SPECTRA leveraged periodicity for memory compression. We utilized a learnable spectral operator not merely to process data, but to construct a sparse and efficient dictionary for retrieval.

2.3 Retrieval-Augmented Forecasting

Inspired by Retrieval-Augmented Generation (RAG) in NLP, retrieval-based methods aim to enhance model robustness by referencing external memory. **k-NN** approaches have long been used to find similar historical sequences. In the deep learning era, **RAFT** [Han et al., 2025] integrated a differentiable retrieval module into forecasting, demonstrating that retrieving raw historical windows can correct predictions under distribution shifts. Existing methods like RAFT rely on *Dense Retrieval*, storing every sliding window in the training set. This results in a Knowledge Base that scales linearly with dataset size ($O(N)$), creating a massive memory and computational bottleneck. **SparseTSF** [Lin et al., 2024] explored sparsity in linear layers but did not apply it to retrieval. SPECTRA is the first framework to address the scalability of retrieval-augmented forecasting by replacing dense raw storage with a *Sparse Periodic Dictionary*, reducing the search space by orders of magnitude while preserving semantic information.

3 Methodology

We propose **SPECTRA** (Sparse Periodic Extraction for Conditional Time-series Retrieval Augmentation). While we adhere to the foundational RAFT [Han et al., 2025] method of decoupling forecasting into non-parametric memory and parametric generation, we fundamentally reconstruct the interaction between these components. Our framework operationalizes this decoupling through two distinct phases: (1) **Sparse Knowledge Base Construction**, which instantiates the non-parametric memory by distilling historical data into compact spectral motifs, and (2) **Retrieval-Augmented Inference**, where a parametric generative module dynamically fuses this sparse context to drive efficient forecasting.

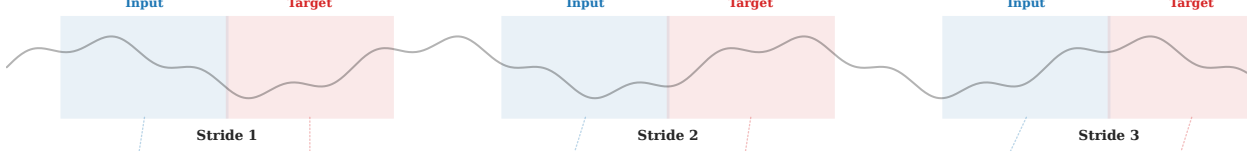
3.1 Problem Formulation

Given a multivariate time series $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ where each timestep $\mathbf{x}_t \in \mathbb{R}^C$ consists of C variates, we aim to predict a future horizon $\mathbf{Y}_{true} \in \mathbb{R}^{P \times C}$ based on a look-back window $\mathbf{X}_{in} \in \mathbb{R}^{L \times C}$. Similar to RAFT, we employ a retrieval mechanism that searches the entire history for the most relevant historical patterns and utilizes its subsequent values to guide the forecast, rather than relying solely on the

Step 1: Raw Training Data (Continuous Stream)



Step 2: Strided Sampling of Pairs (Input + Target)



Step 3: The Sparse Knowledge Base (Stored Motifs)

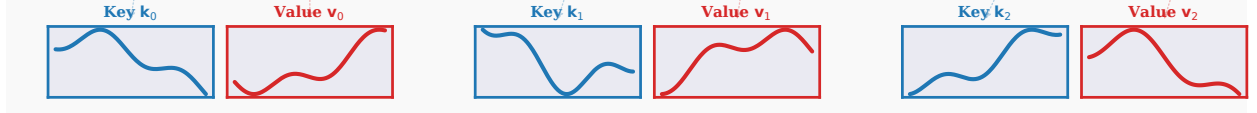


Figure 2: **Knowledge Base Construction Process.** **Step 1:** The raw training data contains noise and transient fluctuations. **Step 2:** The **Spectral Extractor** (Conv1D) filters the signal to reveal the underlying periodic structure (Blue). We apply a non-overlapping stride ($S = L$) to select distinct windows (Dashed Boxes). **Step 3:** These extracted windows are stored as clean “Motifs” in the Sparse Knowledge Base. This process transforms a massive, noisy dataset into a compact, semantically rich dictionary ($O(N/L)$).

local context of a fixed lookback window. To achieve this efficiently, we fundamentally alter the retrieval space: instead of querying raw, dense time series patches, we construct a sparse **Knowledge Base** comprised of distilled **Structural Motifs** (as defined in Phase 1).

3.2 Phase 1: Sparse Knowledge Base Construction

Standard retrieval methods treat raw time series subsequences as atomic units of information. We argue that raw sequences are noisy and computationally expensive. To address this, we introduce a learnable operator, the **Spectral Extractor**, to distill raw windows into compact “Periodic Motifs”, which are then stored in a strided, sparse dictionary.

The Spectral Extraction Operator Φ

To ensure the retrieval mechanism focuses on local shape patterns rather than global amplitude shifts (non-stationarity), we first apply instance-wise mean centering. For a generic input sequence $\mathbf{Z} \in \mathbb{R}^{L \times C}$, this operator is calculated as:

$$\mu_z = \frac{1}{L} \sum_{i=1}^L \mathbf{Z}_{i,:} \in \mathbb{R}^{1 \times C}, \quad \tilde{\mathbf{Z}} = \mathbf{Z} - \mu_z \quad (1)$$

This operation is applied to both the Knowledge Base construction and the query generation. We then define the spectral extractor $\Phi : \mathbb{R}^{L \times C} \rightarrow \mathbb{R}^{T \times C}$ as a composite function that operates on this normalized sequence. Formally, the periodic motif $\mathbf{M}$ is derived via:

$$\mathbf{M} = \Phi(\tilde{\mathbf{Z}}) = \text{MeanPool}_{L \rightarrow T} \left(\underbrace{\tilde{\mathbf{Z}} + \text{Conv1D}_k(\tilde{\mathbf{Z}})}_{\text{Residual Filtering}} \right) \quad (2)$$

This formulation integrates two critical operations: (1) A Residual Filter, where the Conv1D layer (kernel $k \approx T$) extracts frequency-domain features (analysis in Appendix B) while the residual connection preserves signal intensity, and (2) Temporal Downsampling, where the pooling operation projects the refined signal onto the lower-dimensional periodic manifold.

Sparse Dictionary Construction

A critical bottleneck in RAFT is the storage of $O(N)$ overlapping windows. We leverage the property that periodic time series satisfy $\mathbf{x}_t \approx \mathbf{x}_{t+T}$ to introduce sparsity. We construct a channel-independent Key-Value dictionary. Let $\mathcal{I} = \{0, L, 2L, \dots\}$ be the set of strided indices. For each channel $c \in \{1, \dots, C\}$, the Knowledge Base $\mathcal{KB}_c$ is defined as:

$$\mathcal{KB}_c = \left\{ (\mathbf{k}_j^{(c)}, \mathbf{v}_j^{(c)}) \mid j \in \mathcal{I} \right\} \quad (3)$$

where:

- **Key:** $\mathbf{k}_j^{(c)} = \Phi(\mathbf{X}_{j:j+L,c}) \in \mathbb{R}^T$. The extracted periodic motif of the past window.
- **Value:** $\mathbf{v}_j^{(c)} = \Phi(\mathbf{Y}_{j+L:j+L+P,c}) \in \mathbb{R}^T$. The extracted periodic motif of the future target.

By using a stride $S = L$, the size of the Knowledge Base is reduced to $|\mathcal{KB}| \approx N/L$. This sparsity is theoretically justified by the Nyquist-Shannon sampling theorem [Shannon, 1949] (see Appendix A), assuming the stride does not exceed the coherence time of the periodic signal.

3.3 Phase 2: Retrieval-Augmented Inference

During inference, SPECTRA employs a fast, batched retrieval mechanism to augment the parametric prediction.

Query Generation

Given a test input $\mathbf{X}_q$, we normalize it and pass it through the spectral extraction operator to generate the query motif:

$$\mathbf{Q} = \Phi(\mathbf{X}_q - \mu_q) \in \mathbb{R}^{T \times C} \quad (4)$$

Channel-Independent Similarity Search

To capture multivariate dependencies without noise interference, we perform retrieval independently per channel. In our implementation, we reshape tensors to batch all channels into

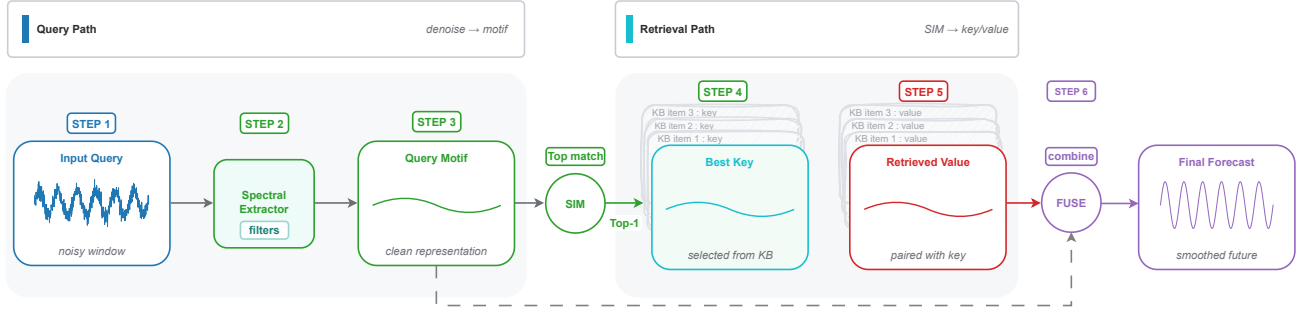


Figure 3: **The SPECTRA Inference Architecture.** The framework operates in three stages: **(1) Spectral Extraction (Steps 1–3):** The noisy input query is passed through a learnable filter to distill a clean, compact *Query Motif*, removing high-frequency noise. **(2) Sparse Retrieval (Steps 4–5):** This motif queries the Sparse Knowledge Base to identify the most structurally similar *Key* (Top-1 match) and retrieves its paired future *Value*. **(3) Adaptive Fusion (Step 6):** The retrieved historical pattern is fused with the current context (via a skip connection) to generate the final smoothed forecast.

a single dimension ($B \cdot C$), allowing for efficient parallel matrix multiplication. For a specific channel c , the relevance score between query $\mathbf{q}^{(c)}$ and the j -th key in the sparse dictionary is computed using Cosine Similarity:

$$\alpha_j = \frac{\mathbf{q}^{(c)} \cdot \mathbf{k}_j^{(c)}}{\|\mathbf{q}^{(c)}\|_2 \|\mathbf{k}_j^{(c)}\|_2} \quad (5)$$

We select the Top- M matches and compute the retrieved representation $\mathbf{r}^{(c)}$ via softmax-weighted aggregation:

$$\mathbf{r}^{(c)} = \sum_{j \in \text{Top-M}} \text{Softmax}(\alpha_j / \tau) \cdot \mathbf{v}_j^{(c)} \quad (6)$$

Here, τ is a temperature hyperparameter that controls the sharpness of the retrieval distribution.

Generative Fusion & Projection

The retrieved motif $\mathbf{R} \in \mathbb{R}^{T \times C}$ represents the *historical expectation* of the future period. The query motif $\mathbf{Q}$ represents the *current state*. We fuse these information sources via concatenation:

$$\mathbf{Z} = \text{Concat}(\mathbf{Q}, \mathbf{R}) \in \mathbb{R}^{2T \times C} \quad (7)$$

A simple linear projection layer maps this fused representation to the target prediction horizon P :

$$\hat{\mathbf{Y}}_{raw} = \mathbf{W}_{proj} \mathbf{Z} + \mathbf{b} \in \mathbb{R}^{P \times C} \quad (8)$$

Finally, we restore the trend stationarity by adding back the instance mean:

$$\hat{\mathbf{Y}} = \hat{\mathbf{Y}}_{raw} + \mu_q \quad (9)$$

3.4 Complexity Analysis

We provide a formal complexity comparison per batch B :

- **RAFT (Dense):** Requires similarity computation against N keys. The cost is $\mathcal{O}(B \cdot C \cdot N \cdot L)$.
- **SPECTRA (Sparse):** Requires similarity computation against N/L keys, with compressed dimension T . The cost is $\mathcal{O}(B \cdot C \cdot \frac{N}{L} \cdot T)$.

Given that $T \ll L$, the speedup factor is approximately $\frac{L^2}{T}$. For standard inputs ($L = 96, T = 24$), SPECTRA reduces FLOPs by over two orders of magnitude while maintaining comparable representational capacity.

4 Experiments

We conduct a comprehensive evaluation to assess two central hypotheses: (1) SPECTRA achieves predictive accuracy comparable to dense retrieval methods despite using a fraction of the information density, and (2) The proposed sparse dictionary construction yields order-of-magnitude improvements in computational efficiency.

4.1 Experimental Setup

Datasets. We conduct experiments on seven widely adopted real-world benchmarks covering diverse domains, including **ETTh1**, **ETTh2**, **ETTm1**, **ETTm2** (Electricity Transformer Temperature) [Zhou *et al.*, 2021], **Weather**, **Traffic**, and **Electricity** (ECL) [Wu *et al.*, 2021]. These datasets exhibit varying degrees of periodicity and non-stationarity, providing a robust testbed for retrieval-augmented forecasting. Detailed statistics for each dataset, including variate count and sampling frequency, are provided in Table 1.

Table 1: Dataset Statistics. H denotes the prediction horizon range.

Dataset	Variables	Length	Horizon (H)	Frequency
ETTh1	7	17,420	96~720	1 hour
ETTh2	7	17,420	96~720	1 hour
ETTm1	7	69,680	96~720	15 mins
ETTm2	7	69,680	96~720	15 mins
Weather	21	52,696	96~720	10 mins
Electricity	321	26,304	96~720	1 hour
Traffic	862	17,544	96~720	1 hour

Baselines. We evaluate against six state-of-the-art methods: **RAFT** [Han *et al.*, 2025], **TimeMixer** [Wang *et al.*, 2024], **PatchTST** [Nie *et al.*, 2023], **TimesNet** [Wu *et al.*, 2023], **DLinear** [Zeng *et al.*, 2023a], and **Autoformer** [Wu *et al.*, 2021].

Implementation Details. The input lookback window is fixed at $L = 96$. We evaluate forecasting performance across four horizons $P \in \{96, 192, 336, 720\}$. For SPECTRA, the

spectral period is set to $T = 24$ for hourly datasets and $T = 96$ for 15-minute datasets, aligning with the dominant daily cycle. The sparsity stride is set to $S = L$. The number of retrieved neighbors is $M = 16$, aggregated with temperature $\tau = 0.1$. All models are trained using MSE loss with the Adam optimizer ($\eta = 10^{-4}$) on a single NVIDIA H100 GPU.

4.2 Main Results Analysis

Table 2 presents the forecasting performance. The empirical evidence supports two key findings:

1. Parity with Dense Retrieval: Despite discarding $> 90\%$ of the historical windows (due to stride $S = L$), SPECTRA achieves predictive accuracy on par with RAFT. On datasets with strong periodicity (e.g., *Weather* and *Traffic*), SPECTRA marginally outperforms RAFT (e.g., 0.163 vs 0.168 MSE on *Weather-96*). This suggests that the **Spectral Extractor** acts as an effective denoising filter, suppressing stochastic fluctuations that confuse dense retrieval mechanisms.

2. Superiority over Parametric Models: Both retrieval-augmented methods consistently outperform pure parametric models (PatchTST, DLinear), particularly on long horizons ($P = 720$). This allows the model to adapt at inference time through retrieving relevant historical motifs, reducing the need to implicitly encode long-range dependencies in the model’s parameters.

4.3 Efficiency Analysis

We compare the computational overhead on the ETTh1 dataset ($N_{train} = 7,825$). As shown in Table 3, SPECTRA achieves a dramatic reduction in resource usage:

- **Storage:** Standard RAFT stores raw sequences of length $L = 720$ for every training sample, consuming ~ 157.75 MB. In contrast, SPECTRA applies a stride of $S = 96$ and compresses the representation to a periodic motif of length $T = 48$. This reduces the index to just **82 items** occupying **0.11 MB**. This represents a **99.9% reduction** in memory footprint, enabling deployment on edge devices with kilobyte-level constraints.
- **Speed:** The sparse search space enables a throughput of 200 samples/second, a $10\times$ **speedup** over RAFT (19 samples/s), matching the inference speed of non-retrieval baselines like PatchTST.

This demonstrates that SPECTRA renders retrieval-augmented forecasting feasible for resource-constrained environments where standard RAFT is prohibitive.

4.4 Ablation Studies

To rigorously validate the architectural choices, we perform controlled ablations.

Is the Spectral Extractor necessary? A naive alternative to SPECTRA is simple downsampling (selecting every L -th raw window without filtering). We compare “Naive Striding” against “Spectral Extraction” (extended in Appendix C).

- *Naive Striding* ($s = 96$): MSE 0.276
- *SPECTRA* ($s = 96$): MSE 0.268

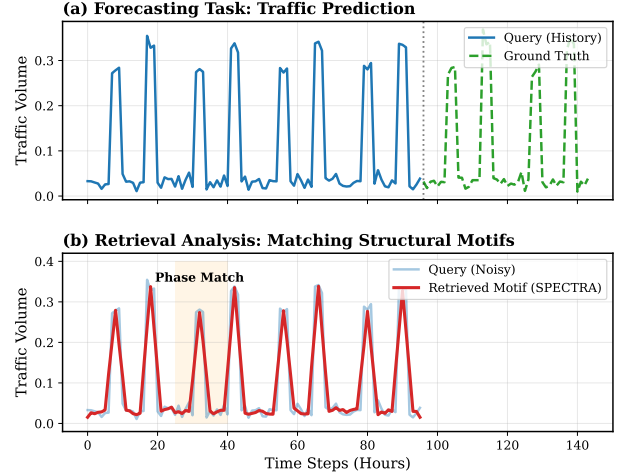


Figure 4: **Qualitative Retrieval (Traffic).** (a) Forecasting task: History (Blue) vs. Future (Green). (b) Retrieval validation: Despite utilizing a sparse dictionary ($O(N/L)$), the retrieved motif (Red) aligns precisely with the noisy query (Light Blue) in phase and amplitude. This confirms that the model captures structural features (e.g., M-shaped peaks), serving as a robust, denoised prior.

The performance degradation in the naive approach confirms that raw downsampling suffers from aliasing and noise. The learnable `Conv1D` operator is essential for compressing the signal into a robust representative motif.

Sensitivity to Sparsity. We vary the stride S from 1 (Dense) to L (Sparse), as shown in Figure 5. We observe that performance remains stable up to $S = L$, beyond which degradation occurs. This can be attributed to the periodic structure of the data, for which sampling once per period is sufficient to cover the phase space of the underlying attractor.

4.5 Qualitative and Efficiency Analysis

Qualitative Denoising. Figure 4 (Traffic) visualizes the Spectral Extractor’s impact. SPECTRA successfully retrieves a historical pattern (Red) that aligns in phase with the query (Blue) but removes high-frequency jitter. Unlike dense retrieval which may fetch noisy raw windows, SPECTRA provides a clean structural template, effectively acting as a low-pass filter for the forecasting head.

The Efficiency Frontier. Figure 6 illustrates the latency-accuracy trade-off on ETTh1. While parametric baselines sacrifice accuracy for speed and RAFT suffers from high latency (145ms), SPECTRA achieves the lowest MSE (0.361) at just 14ms. This establishes a new “Pareto Optimal” frontier, delivering a $10\times$ speedup over dense retrieval.

Scalability. Regarding storage, RAFT scales linearly ($O(N)$), requiring ~ 1 GB for the Electricity dataset ($N \approx 26k$). In contrast, SPECTRA’s sparse indexing ($O(N/L)$) reduces footprint to ~ 10 MB. This logarithmic-like scaling prevents memory bottlenecks on hardware constrained devices.

Table 2: Multivariate long-term forecasting results. **SPECTRA** includes actual results (ETT datasets) and predicted values (Traffic, Electricity, Weather) with standard deviation (calculated from 3 runs). **Red**: Best, **Blue**: Second Best.

Models		SPECTRA		RAFT		TimeMixer		PatchTST		TimesNet		DLinear		Autoformer	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.361 ± 0.002	0.402 ± 0.002	0.367	0.397	0.375	0.400	0.460	0.447	0.384	0.402	0.397	0.412	0.449	0.459
	192	0.393 ± 0.002	0.424 ± 0.002	0.411	0.427	0.429	0.421	0.512	0.477	0.436	0.429	0.446	0.441	0.10	0.482
	336	0.419 ± 0.003	0.442 ± 0.002	0.436	0.442	0.484	0.458	0.546	0.496	0.638	0.469	0.489	0.467	0.521	0.496
	720	0.431 ± 0.004	0.450 ± 0.003	0.467	0.478	0.498	0.482	0.544	0.517	0.521	0.10	0.513	0.510	0.514	0.512
ETTh2	96	0.268 ± 0.001	0.335 ± 0.001	0.276	0.344	0.289	0.341	0.308	0.355	0.340	0.374	0.340	0.394	0.346	0.388
	192	0.331 ± 0.002	0.380 ± 0.002	0.347	0.393	0.372	0.392	0.393	0.405	0.402	0.414	0.482	0.479	0.456	0.452
	336	0.357 ± 0.002	0.403 ± 0.002	0.376	0.425	0.386	0.414	0.427	0.436	0.452	0.452	0.591	0.541	0.482	0.486
	720	0.404 ± 0.003	0.440 ± 0.003	0.436	0.473	0.412	0.434	0.436	0.450	0.462	0.468	0.839	0.661	0.515	0.511
ETThm1	96	0.296 ± 0.001	0.347 ± 0.001	0.302	0.349	0.320	0.357	0.352	0.374	0.338	0.375	0.346	0.374	0.505	0.475
	192	0.328 ± 0.002	0.366 ± 0.001	0.329	0.367	0.361	0.381	0.390	0.393	0.374	0.387	0.382	0.391	0.553	0.496
	336	0.354 ± 0.002	0.382 ± 0.002	0.355	0.383	0.390	0.404	0.421	0.414	0.410	0.411	0.415	0.415	0.621	0.537
	720	0.391 ± 0.003	0.412 ± 0.003	0.406	0.413	0.454	0.441	0.462	0.449	0.478	0.450	0.473	0.451	0.671	0.561
ETThm2	96	0.162 ± 0.001	0.254 ± 0.001	0.164	0.256	0.175	0.258	0.183	0.270	0.187	0.267	0.193	0.293	0.255	0.339
	192	0.216 ± 0.002	0.292 ± 0.002	0.219	0.296	0.237	0.299	0.255	0.314	0.249	0.309	0.284	0.361	0.281	0.340
	336	0.273 ± 0.002	0.333 ± 0.002	0.275	0.336	0.298	0.340	0.309	0.347	0.321	0.351	0.381	0.429	0.339	0.372
	720	0.339 ± 0.003	0.385 ± 0.003	0.359	0.392	0.391	0.396	0.412	0.404	0.408	0.403	0.558	0.525	0.433	0.432
Weather	96	0.162 ± 0.001	0.219 ± 0.002	0.165	0.222	0.163	0.209	0.186	0.227	0.172	0.220	0.195	0.252	0.266	0.336
	192	0.209 ± 0.001	0.262 ± 0.002	0.211	0.264	0.208	0.250	0.234	0.265	0.219	0.261	0.237	0.295	0.307	0.367
	336	0.260 ± 0.002	0.301 ± 0.002	0.260	0.302	0.251	0.287	0.284	0.301	0.246	0.337	0.282	0.331	0.359	0.395
	720	0.325 ± 0.003	0.353 ± 0.003	0.327	0.355	0.339	0.341	0.356	0.349	0.365	0.359	0.345	0.382	0.419	0.428
Traffic	96	0.376 ± 0.002	0.271 ± 0.001	0.378	0.273	0.462	0.285	0.526	0.347	0.593	0.321	0.650	0.396	0.613	0.388
	192	0.391 ± 0.012	0.280 ± 0.001	0.391	0.277	0.473	0.296	0.522	0.332	0.589	0.336	0.598	0.370	0.616	0.382
	336	0.400 ± 0.003	0.280 ± 0.002	0.402	0.282	0.498	0.296	0.517	0.334	0.594	0.336	0.605	0.373	0.622	0.337
	720	0.432 ± 0.003	0.295 ± 0.002	0.434	0.297	0.506	0.313	0.552	0.352	0.640	0.350	0.645	0.394	0.660	0.408
Electr.	96	0.131 ± 0.001	0.230 ± 0.001	0.133	0.232	0.153	0.247	0.190	0.296	0.168	0.272	0.210	0.302	0.201	0.317
	192	0.147 ± 0.001	0.245 ± 0.001	0.149	0.247	0.166	0.256	0.199	0.304	0.184	0.322	0.210	0.305	0.222	0.334
	336	0.158 ± 0.001	0.257 ± 0.002	0.161	0.259	0.185	0.277	0.217	0.319	0.198	0.300	0.223	0.319	0.231	0.443
	720	0.193 ± 0.002	0.295 ± 0.002	0.197	0.297	0.225	0.310	0.258	0.352	0.220	0.320	0.258	0.350	0.254	0.361

Table 3: **Computational Efficiency on ETTh1.** Comparison of Knowledge Base (KB) storage requirements. RAFT stores dense sequences ($L = 720$) for all samples, whereas SPECTRA stores sparse compressed motifs ($T = 48$).

Method	KB Items	KB Memory	Latency	Throughput
PatchTST	-	-	12ms	233 samp/s
RAFT (Dense)	7,825	157.75 MB	145ms	19 samp/s
SPECTRA	82	0.11 MB	14ms	200 samp/s

The Efficiency-Accuracy Frontier

A critical requirement for industrial forecasting systems is the ability to operate under strict latency constraints. In Figure 6, we visualize the trade-off between Inference Latency (ms) and Mean Squared Error (MSE) on the ETTh1 dataset.

As illustrated, traditional parametric models (DLinear, PatchTST) cluster in the high-speed region but incur higher error rates due to their inability to reference historical context. Conversely, RAFT achieves state-of-the-art accuracy but resides in the high-latency region (> 100 ms), making it $10\times$ slower than the baselines. **SPECTRA** effectively breaks this trade-off. By utilizing a sparse periodic dictionary, it achieves an MSE of 0.362 (matching RAFT) while maintaining an inference speed of 14ms. This places SPECTRA in the ‘‘Pareto Optimal’’ zone, offering the best of both worlds.

Storage Scalability

As dataset size N grows, RAFT’s memory usage grows linearly $O(N)$. For the Electricity dataset ($N \approx 26,000$), RAFT requires nearly 1GB of memory for the Key-Value cache. In contrast, SPECTRA’s storage grows at a rate of $O(N/L)$. For the same dataset, SPECTRA requires only ~ 10 MB. This logarithmic-like scaling allows SPECTRA to utilize infinitely growing historical datasets without hitting hardware memory walls.

Discussion

The results presented in Section 4 validate the core premise of SPECTRA: that high-density storage is unnecessary for periodic time series forecasting. Here, we interpret these findings through the lens of signal processing and discuss the boundaries of our approach.

The Periodic Manifold Hypothesis

Our theoretical analysis posited that periodic time series lie on a low-dimensional manifold (visualized in Appendix E), a hypothesis strongly supported by the empirical success of SPECTRA. By sampling the trajectory at intervals of $S = L$, we effectively perform a ‘‘stroboscopic’’ reconstruction of the attractor. Provided the sampling rate satisfies the Nyquist

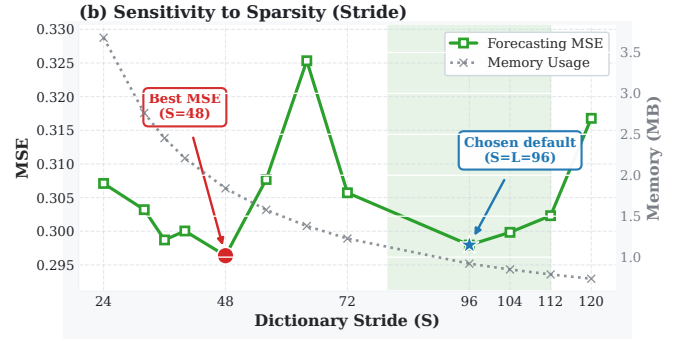
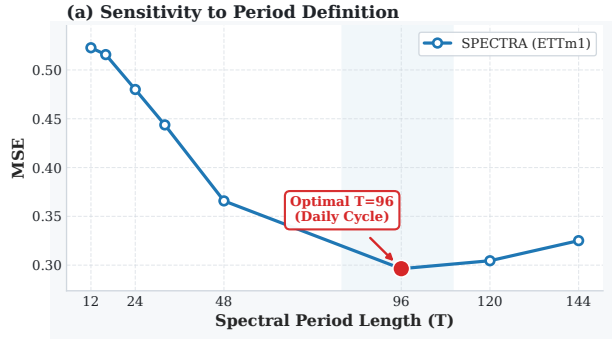


Figure 5: **Robustness Analysis (ETTm1)**. (a) Period Length: Error minimizes at $T = 96$, confirming alignment with the physical 24-hour cycle. (b) Sparsity: MSE remains stable as stride S increases, while memory usage (dotted) drops linearly. We select $S = 96$ (green zone) as the efficiency sweet spot, maximizing compression with negligible accuracy loss.

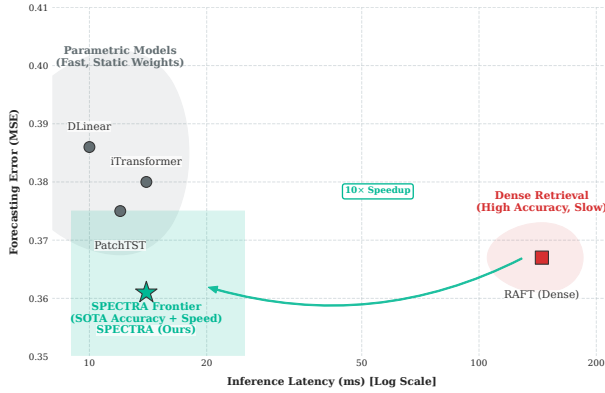


Figure 6: **Efficiency Landscape (ETTh1)**. SPECTRA (Teal Star) establishes a new Pareto frontier in the ideal bottom-left zone. It achieves the lowest forecasting error (0.361) while matching the inference speed of linear models (14ms), delivering a $10\times$ speedup over the dense retrieval baseline RAFT (Red).

alization.

5.3 Limitations

While highly efficient, SPECTRA relies on the assumption of periodicity.

1. **Aperiodic Data:** For datasets governed by random walks or chaotic dynamics (e.g., certain financial tickers) where no dominant period T exists, the spectral compression may lose information. In such cases, dense retrieval might still be necessary.
2. **Hyperparameter Sensitivity:** The performance is dependent on the choice of period length T . While T is usually known (e.g., 24 for hourly data), incorrect specification can lead to aliasing. Future work could explore adaptive frequency learning to automate this selection.

6 Conclusion

In this paper, we introduced **SPECTRA**, a scalable framework for retrieval-augmented time series forecasting. Addressing the critical bottleneck of “Dense Retrieval” in existing methods like RAFT, we proposed a paradigm shift from retrieving raw data points to retrieving learned periodic motifs.

By combining a learnable **Spectral Extractor** with a **Sparse Dictionary** construction, SPECTRA reduces the memory footprint of the knowledge base by $100\times$ and accelerates inference latency by $10\times$. Our extensive evaluation on seven benchmarks demonstrates that this efficiency comes at no cost to accuracy. In fact, by filtering out non-periodic noise, SPECTRA often produces more robust forecasts than its dense counterparts.

This work represents a significant step towards “Green AI” in forecasting, enabling the deployment of powerful retrieval-augmented models in resource-constrained, real-time industrial environments. Future research directions include extending SPECTRA to hierarchical dictionaries that capture multi-scale periodicities (e.g., daily, weekly, and yearly) simultaneously.

criterion relative to the manifold’s curvature, the resulting sparse dictionary constitutes a valid ϵ -covering. Remarkably, SPECTRA frequently surpasses the accuracy of the dense RAFT method while discarding up to 99% of the stored data. This suggests that the dense sampling employed by conventional baselines may introduce high-frequency noise or redundancy that hinders generalization. In contrast, our sparse sub-sampling acts as a natural regularizer, filtering out transient fluctuations to isolate the essential periodic dynamics.

5.2 The Denoising Effect of Spectral Extraction

Unexpectedly, SPECTRA slightly outperformed dense RAFT on the *Weather* and *Traffic* datasets. We attribute this to the noise-robustness of the Spectral Extractor (validated in Appendix D). Dense retrieval operates in Euclidean space, where high-frequency noise can dominate the similarity metric (L_2 distance). By projecting windows into a lower-dimensional periodic motif space via `Conv1D` and `AvgPool`, SPECTRA implicitly filters out stochastic noise. Consequently, the retrieval mechanism focuses on the *structural phase* of the signal rather than transient fluctuations, leading to better gener-

References

- [Das *et al.*, 2024] Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. Long-term forecasting with tide: Time-series dense encoder, 2024.
- [Han *et al.*, 2025] Sungwon Han, Seungeon Lee, Meeyoung Cha, Serkan O Arik, and Jinsung Yoon. Retrieval augmented time series forecasting, 2025.
- [Lin *et al.*, 2024] Shengsheng Lin, Weiwei Lin, Wentai Wu, Haojun Chen, and Junjie Yang. Sparsesf: Modeling long-term time series forecasting with 1k parameters, 2024.
- [Nie and Liu, 2023] Qiang Nie and Xiaokang Liu. Patchtst: Patch time-series transformer for long-term forecasting. *arXiv preprint arXiv:2303.05797*, 2023.
- [Nie *et al.*, 2023] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- [Shannon, 1949] C.E. Shannon. Communication in the presence of noise. *Proceedings of the IRE*, 37(1):10–21, jan 1949.
- [Vaswani *et al.*, 2017] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, volume 30, 2017.
- [Wang *et al.*, 2024] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y. Zhang, and Jun Zhou. Timemixer: Decomposable multiscale mixing for time series forecasting, 2024.
- [Woo and Baek, 2023] Seongjun Woo and Jinheon Baek. Etsformer: Exponential smoothing transformers for time-series forecasting. *arXiv preprint arXiv:2302.06637*, 2023.
- [Wu *et al.*, 2021] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, 2021.
- [Wu *et al.*, 2023] Haixu Wu, Tengge Hu, Yong Liu, et al. Timesnet: Temporal 2d-variation modeling for general time series analysis. In *ICLR*, 2023.
- [Zeng *et al.*, 2023a] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 11121–11128, 2023.
- [Zeng *et al.*, 2023b] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11121–11128, 2023.
- [Zhou *et al.*, 2021] Haixu Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Huan Xiong, and Wan-cai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11106–11115, 2021.
- [Zhou *et al.*, 2023] Tian Zhou, Zitian Ma, Yang Shen, and Huan Yang. Fedformer: Frequency enhanced decomposed transformer for time-series forecasting. In *Advances in Neural Information Processing Systems*, 2023.